

Material Design 3 for Web Design

Executive summary

Material Design 3 is best understood on the web as a **token-driven design system** first and a component library second. The official specification now lives primarily at **m3.material.io**, while the official canonical web implementation is **Material Web** (`@material/web`) on **material-web.dev** and GitHub. Material Web is the most direct expression of MD3 tokens on the web, but its docs and repository currently state that **MWC is in maintenance mode pending new maintainers**. Meanwhile, the older **MDC-Web** repository was **archived on January 13, 2025**, so it should be treated as legacy for new work. ¹

For framework choices, the practical landscape is uneven. **Material Web** is the strongest option if you want the most canonical MD3 token model and framework-agnostic web components. **Angular Material** is a very strong Angular-native option, with active theming support, CSS/system-variable generation, accessibility commitments, and test harnesses. **MUI** is excellent for React ergonomics, theming, and ecosystem depth, but its current docs explicitly state that **Material UI currently adopts Material Design 2**, not native MD3. **Vuetify** is highly relevant for Vue because of its productivity, theme system, layout features, and treeshaking, but it is not the canonical Google MD3 reference implementation. ²

If your goal is to become an expert who can apply MD3 to **any** web project, the winning strategy is:

1. Master the **official MD3 foundations**: tokens, color roles, tonal palettes, typography roles, shape, elevation, interaction states, motion, and adaptive layout. ³
2. Implement those foundations as a **token layer in CSS custom properties**. Material Web explicitly models web tokens as CSS custom properties, and MUI plus Angular Material now also lean heavily into CSS variables and color schemes. ⁴
3. Treat any library as an **adapter** over your token system, not your design system itself. This is the most robust way to survive framework migrations, visual refreshes, and bundle/performance constraints. This is an expert recommendation based on the official token architectures and theming APIs across Material Web, Angular Material, and MUI. ⁵

The preferred framework in your request is **unspecified**, so the report stays framework-agnostic by default while providing concrete examples for **vanilla web components, React, Angular, and Vue**. ⁶

The official MD3 web model

Material describes itself as an adaptable system of **guidelines, components, and tools** backed by open-source code, and its current web-facing foundation is organized around **design tokens**. The official docs explicitly describe tokens as the building blocks of UI elements, used consistently across design, tools, and code. On the web, Material Web states that these tokens are expressed as **CSS custom properties**. ⁷

Token hierarchy and why it matters

The most important expert-level concept is the **three-level token stack**:

- **Reference tokens** hold concrete values such as font families or raw palette values.
- **System tokens** express design decisions and roles like color, typescale, shape, elevation, and motion.
- **Component tokens** map those design decisions to concrete component parts. ⁸

Material Web makes this hierarchy explicit and notes that **each component token maps to a system token, which has a concrete reference value**. That is the mental model to keep in your head when building any web theme, regardless of framework. ⁹

```
<svg width="760" height="180" viewBox="0 0 760 180" xmlns="http://www.w3.org/2000/svg" role="img"
aria-label="Material token hierarchy diagram"> <rect x="20" y="50" width="180" height="80" rx="14"
fill="#f3f4f6" stroke="#9ca3af"/> <text x="110" y="80" text-anchor="middle" font-family="Arial, sans-serif"
font-size="18">Reference tokens</text> <text x="110" y="105" text-anchor="middle" font-family="Arial, sans-
serif" font-size="13">Raw values</text> <text x="110" y="123" text-anchor="middle" font-family="Arial, sans-
serif" font-size="13">fonts, palette tones</text>
```

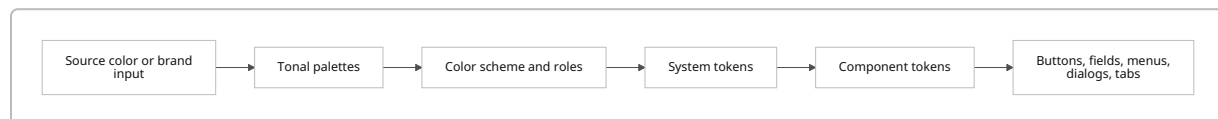
```
<rect x="290" y="50" width="180" height="80" rx="14" fill="#eef6ff" stroke="#60a5fa"/> <text x="380" y="80"
text-anchor="middle" font-family="Arial, sans-serif" font-size="18">System tokens</text> <text x="380"
y="105" text-anchor="middle" font-family="Arial, sans-serif" font-size="13">Roles and decisions</text> <text
x="380" y="123" text-anchor="middle" font-family="Arial, sans-serif" font-size="13">color, type, shape</text>
```

```
<rect x="560" y="50" width="180" height="80" rx="14" fill="#f0fdf4" stroke="#34d399"/> <text x="650"
y="80" text-anchor="middle" font-family="Arial, sans-serif" font-size="18">Component tokens</text> <text
x="650" y="105" text-anchor="middle" font-family="Arial, sans-serif" font-size="13">Applied to parts</text>
<text x="650" y="123" text-anchor="middle" font-family="Arial, sans-serif" font-size="13">button, dialog,
tab</text>
```

```
<line x1="200" y1="90" x2="290" y2="90" stroke="#6b7280" stroke-width="2"/> <polygon points="290,90
278,84 278,96" fill="#6b7280"/> <line x1="470" y1="90" x2="560" y2="90" stroke="#6b7280" stroke-
width="2"/> <polygon points="560,90 548,84 548,96" fill="#6b7280"/> </svg>
```

This hierarchy is the cleanest way to separate **brand**, **system behavior**, and **component implementation**. It is also why token-first MD3 implementations are easier to migrate than style-overriding-heavy systems.

¹⁰



The diagram above mirrors the official MD3 color and token model: a source color produces palettes and schemes, which become system roles and then component-level decisions. ¹¹

Color, tonal palettes, typography, shape, and elevation

MD3 color is not a random set of branded hex values. The official docs describe a system where a source color produces a **core palette** made of **five tonal palettes**, and where **colors from those palettes are assigned to roles** such as primary, secondary, tertiary, error, background, surface, outline, and their “on-” companions. Material’s official color pages also emphasize that role pairings are designed to produce accessible relationships and that the system supports multiple contrast levels. ¹²

Material Web exposes those roles with `--md-sys-color-*` tokens and explicitly notes that every color token has a corresponding `--md-sys-color-on-*` content color for accessible contrast. That means the expert move is to work with **roles**, not hard-coded per-component colors. ¹³

Typography in MD3 is organized around five role families — **display, headline, title, body, label** — and Material Web exposes them as `--md-sys-typescale-*` tokens. It also distinguishes **brand** and **plain** typefaces through `--md-ref-typeface-*` tokens. ¹⁴

Shape is expressed as a **shape scale** that controls roundedness through system tokens such as `--md-sys-shape-corner-small`, `medium`, `large`, and `full`. Shape in MD3 is not decoration; the official docs explicitly frame shape as a way to direct attention, communicate state, and express brand. ¹⁵

Elevation in MD3 is more nuanced than “add box-shadow.” The official docs say elevation can be depicted with **shadows, surface tone differences**, and **scrim**s, and specifically recommend using shadows only where they help separate a surface from its background. On the web, this means you should think in terms of **elevated surfaces**, not just shadow presets. ¹⁶

Interaction states, motion, and adaptive layout

The official MD3 state model includes **hover, focus, press, drag, and disabled**. Material’s state-layer guidance gives concrete overlay opacities: **hover 8%, focus 10%, press 10%, drag 16%**. Disabled components are explicitly described as not focusable, draggable, or pressable. ¹⁷

MD3 motion now has a more expressive, physics-based model. The official docs describe tokens for **spatial movement** and **effects**, each with **fast, default, and slow** variants. Material’s easing/duration pages and motion specs also show that expressive motion uses tokenized curves and durations. The important web caveat is that **Material Web does not currently support** `--md-sys-motion` tokens, so you often have to implement motion policy in app CSS and JS yourself. ¹⁸

For layout, Material’s official adaptive design docs say layout guidance **applies to web**, uses **breakpoints** as opinionated window sizes, and centers on **panes, responsive grids**, and **canonical layouts** such as **feed, list-detail**, and **supporting pane**. Material also describes grids and spacing as the foundation that adapts across breakpoints. ¹⁹

The web library ecosystem

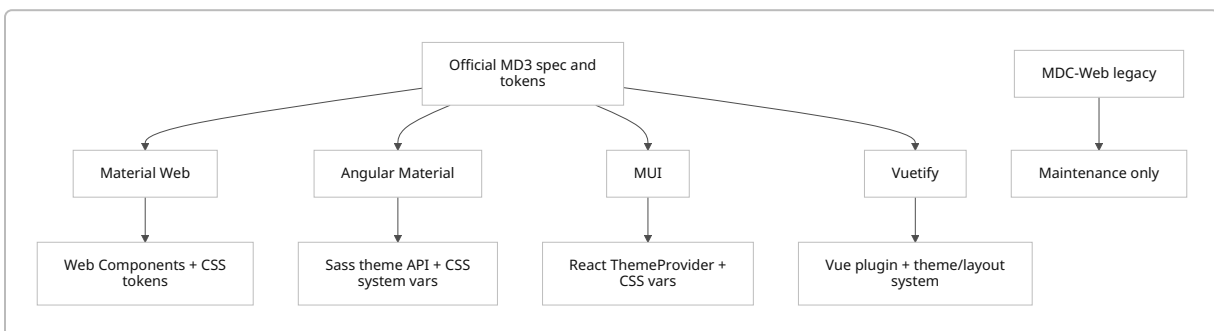
What each library really gives you

The table below is the practical map of the current ecosystem.

Library	What it is	MD3 fidelity	Theming model	Best fit	Main caution
Material Web	Official web component library for MD3. Docs/repo describe it as <code>@material/web</code> , a library of web components for accessible web apps. ²⁰	Highest canonical fidelity to official MD3 tokens on the web. ²¹	CSS custom properties for ref/system/component tokens. ²²	Framework-agnostic apps, design-system teams, custom-element-friendly stacks.	Docs/repo say MWC is in maintenance mode pending new maintainers. ²⁰
MDC-Web	Older framework-agnostic Material Components for the web, successor to MD Lite. ²³	Legacy Material implementation, not the current recommended starting point.	Sass/CSS/JS component packages. ²³	Only for maintaining existing systems.	Repo was archived Jan 13, 2025. ²³
MUI v5+	Major React component ecosystem implementing Google's Material Design with strong theming/customization. ²⁴	Current docs explicitly say Material UI supports Material Design 2 and future MD3 work is tracked separately. ²⁵	React <code>ThemeProvider</code> , palette/colorSchemes, CSS variables, optional experimental Pigment CSS path. ²⁶	React teams wanting speed, ecosystem breadth, SSR patterns, and flexible customization.	Not native MD3 today; some APIs like Pigment CSS are still early/on hold. ²⁷

Library	What it is	MD3 fidelity	Theming model	Best fit	Main caution
Angular Material	Official Angular Material components plus CDK and ARIA utilities, maintained by the Angular team at Google. ²⁸	Strong Angular Material path with M3 palette generation and system variables. ²⁹	Sass <code>mat.theme(...)</code> , generated M3 palettes, or generated CSS system variables with <code>light-dark()</code> and contrast overrides. ²⁹	Angular teams wanting first-party support, accessibility, and robust testing via harnesses/CDK.	More Sass-oriented than Material Web; docs around M3 theming are less elegant than the spec itself. This is an inference from the available theming docs and issue traffic. ³⁰
Vuetify	Vue component framework with strong theme, layout, and productivity features. ³¹	Relevant for Material-inspired Vue apps, but not the canonical Google MD3 reference implementation.	Vuetify plugin theme system with built-in light/dark/system themes, Sass/SCSS customization, application-layout features. ³²	Vue teams prioritizing delivery speed and integrated layout primitives.	Fidelity to the exact MD3 spec varies; treat it as a Vue-first framework, not a direct spec mirror. This is an inference from its positioning and docs. ³³

How the libraries relate to each other



That chart reflects the current reality: the spec is singular, but the implementation styles are not. Expert teams do well when they learn the **spec once** and then swap implementation adapters as needed. ³⁴

High-confidence recommendations by stack

If you want the **most canonical MD3 web expression**, choose **Material Web** and build your app shell with native HTML, CSS Grid/Flexbox, and a token layer. If you want **Angular-first production safety**, choose **Angular Material**. If you want **React productivity and ecosystem strength**, choose **MUI**, but treat MD3 as a **design-language customization target** rather than something the library gives you by default. If you want **Vue-first integrated app layout and theme ergonomics**, Vuetify is highly relevant. This recommendation is an inference from the library positioning, official theming models, and documented status of MD3 support. ³⁵

Theming and implementation architecture

The expert mental model

Theming on the web becomes much simpler when you split it into four layers:

1. **Brand input**: source color, fonts, corner character, spacing rhythm.
2. **System tokens**: MD3 roles such as primary, surface, body-medium, corner-large.
3. **Component tokens**: how a button, field, or dialog consumes those roles.
4. **Host app rules**: responsive layout, content density, state handling, and platform quirks. ²²

This is why CSS variables are such a good fit for MD3 on the web. Material Web uses them directly. MUI now supports CSS theme variables and multiple color schemes. Angular Material can generate CSS themes split into color, typography, elevation, shape, and state variables. ⁴

Build-time versus runtime theming

There is no single “best” theming strategy; there are trade-offs.

Strategy	How it works	Advantages	Costs	Strong fit
Build-time token generation	Generate CSS variables during build and ship them with the app.	Simple runtime, predictable SSR, smaller JS overhead.	Less dynamic personalization unless you ship many schemes.	Angular Material generated CSS, Material Web with pre-generated token files. ³⁶
Runtime token injection	Compute or fetch token values on startup and write them to <code>:root</code> .	Supports dynamic color, personalization, per-tenant branding.	More startup work and more care around FOUC/SSR.	Material Color Utilities-based systems, app shells that need live brand switching. ³⁷

Strategy	How it works	Advantages	Costs	Strong fit
Hybrid CSS-variables theming	Build light/dark or named schemes ahead of time; switch by class/data attribute/media query.	Strong SSR story, low runtime cost, good user preference support.	More HTML/CSS output and more scheme coordination.	MUI <code>cssVariables</code> + <code>colorSchemes</code> , Angular Material <code>light-dark()</code> . ³⁸

MUI's own docs describe the trade-off clearly: CSS theme variables can prevent dark-mode SSR flicker and support unlimited color schemes, but for server-side apps the HTML is larger and FCP can be a bit longer because both light and dark variable sets are generated ahead of time. ³⁹

Framework-agnostic token-first pattern

The most portable pattern is to publish your MD3 roles as CSS variables on `:root` or a theme scope.

The official Material Web docs show exactly this kind of usage. ⁴⁰

```
<script type="module">
  import '@material/web/button/filled-button.js';
  import '@material/web/textfield/outlined-text-field.js';
</script>

<style>
  :root {
    /* System color roles */
    --md-sys-color-primary: #6750a4;
    --md-sys-color-on-primary: #ffffff;
    --md-sys-color-surface: #fef7ff;
    --md-sys-color-on-surface: #1d1b20;
    --md-sys-color-outline: #79747e;

    /* System typography roles */
    --md-sys-typescale-title-medium-font: 500 1rem/1.5rem Inter, system-ui,
    sans-serif;
    --md-sys-typescale-body-medium-font: 400 0.875rem/1.25rem Inter, system-ui,
    sans-serif;

    /* System shape roles */
    --md-sys-shape-corner-medium: 12px;
    --md-sys-shape-corner-large: 20px;
  }

  body {
    margin: 0;
  }
</style>
```

```

padding: 2rem;
background: var(--md-sys-color-surface);
color: var(--md-sys-color-on-surface);
font: var(--md-sys-typescale-body-medium-font);
}

.stack {
display: grid;
gap: 1rem;
max-width: 32rem;
}
</style>

<div class="stack">
  <md-outlined-text-field label="Project name"></md-outlined-text-field>
  <md-filled-button>Create project</md-filled-button>
</div>

```

This pattern is the right default when you want your theme to be usable by **custom elements, plain HTML, utility CSS, and framework components at the same time.** ²²

Runtime dynamic-color pattern

Material's official color system and the Material Color Utilities repository both describe dynamic color as a system that generates accessible schemes from input colors via tonal palettes and algorithms. On the web, the safest implementation pattern is usually to generate a token map and write it to the root element at startup. ⁴¹

```

// Framework-agnostic pattern.
// Generate these values server-side or at startup, then inject them.
const md3Tokens = {
  '--md-sys-color-primary': '#006a6a',
  '--md-sys-color-on-primary': '#ffffff',
  '--md-sys-color-primary-container': '#6ff7f6',
  '--md-sys-color-on-primary-container': '#002020',
  '--md-sys-color-surface': '#f4fbfa',
  '--md-sys-color-on-surface': '#161d1d',
  '--md-sys-color-outline': '#6f7978',
};

for (const [name, value] of Object.entries(md3Tokens)) {
  document.documentElement.style.setProperty(name, value);
}

```

If you need **user-personalized dynamic color**, use this runtime pattern. If you need **stability, SSR, and design review predictability**, generate the tokens at build time instead. That recommendation follows

directly from the official token systems and CSS-variable strategies documented by Material Web, Angular Material, and MUI. ⁴²

React with MUI

MUI currently ships a strong React theming system, but not native MD3 defaults. The practical expert move is to use MUI for **React architecture and UX**, then map MD3 concepts into MUI tokens with `ThemeProvider`, `cssVariables`, and `colorSchemes`. MUI's docs recommend `colorSchemes` over the older palette-only mode because it enables automatic mode switching, cross-tab sync, and transition control. ⁴³

```
import * as React from 'react';
import { createTheme, ThemeProvider, CssBaseline, Button, Card } from '@mui/material';

const theme = createTheme({
  cssVariables: true,
  colorSchemes: {
    dark: true,
  },
  shape: {
    borderRadius: 20,
  },
  typography: {
    fontFamily: 'Inter, system-ui, sans-serif',
    h1: { fontWeight: 700 },
    body1: { lineHeight: 1.6 },
  },
  components: {
    MuiButton: {
      defaultProps: { variant: 'contained' },
      styleOverrides: {
        root: {
          borderRadius: 999,
          textTransform: 'none',
        },
      },
    },
    MuiCard: {
      styleOverrides: {
        root: {
          borderRadius: 24,
        },
      },
    },
  },
});
```

```

export default function App() {
  return (
    <ThemeProvider theme={theme} disableTransitionOnChange>
      <CssBaseline />
      <Card sx={{ p: 3, maxWidth: 480 }}>
        <Button>Create project</Button>
      </Card>
    </ThemeProvider>
  );
}

```

Two important MUI trade-offs matter in real projects. First, MUI's `components` theme key is **not tree-shakable**, so very large override objects can bloat your style surface. Second, barrel imports like `@mui/material` hurt development startup and rebuild time; the docs recommend **path imports** instead. ⁴⁴

Angular with Angular Material

Angular Material's current theme-generation path is one of the most interesting in the ecosystem because it bridges official MD3 color science into Angular tooling. The Angular Material custom theme schematic generates M3 palettes using **Material Color Utilities**, can emit **Sass** or **CSS**, supports **high-contrast overrides**, and can generate system variables for **color, typography, elevation, shape, and state**. ²⁹

```

@use '@angular/material' as mat;
@use './theme-colors' as my-theme;

html {
  color-scheme: light dark;

  @include mat.theme((
    color: (
      primary: my-theme.$primary-palette,
      tertiary: my-theme.$tertiary-palette,
    ),
    typography: Roboto,
    density: 0,
  ));

  @media (prefers-contrast: more) {
    @include my-theme.high-contrast-overrides(color-scheme);
  }
}

```

If you do not use Sass, Angular Material's generated CSS themes use `light-dark()` and `color-scheme` to switch between light and dark values, which is a very modern web-native approach.

⁴⁵

Vue with Vuetify

Vuetify is relevant when you want Vue-first productivity and an integrated theme/layout story. Its theme docs describe built-in **light**, **dark**, and **system** themes, and its layout system is one of the more opinionated and productive among the options discussed here. ⁴⁶

```
// vuetify.ts
import 'vuetify/styles';
import { createVuetify } from 'vuetify';

export default createVuetify({
  theme: {
    defaultTheme: 'system',
    themes: {
      light: {
        colors: {
          primary: '#6750a4',
          secondary: '#625b71',
          surface: '#fef7ff',
          background: '#fef7ff',
        },
      },
      dark: {
        colors: {
          primary: '#d0bcff',
          secondary: '#ccc2dc',
          surface: '#141218',
          background: '#141218',
        },
      },
    },
  },
});
```

For Vue teams, the question is not “can Vuetify theme well?” — it clearly can. The question is whether you need **exact MD3 fidelity** or a **fast Vue app framework with Material flavor**. Officially, it is the latter. ³³

Layout, interaction, and accessibility

Responsive layout and grid patterns for the web

The official Material layout system is not just “use breakpoints.” It is about **adaptive panes**, **responsive grids**, and **canonical layout families** that reshape content as space changes. Material says breakpoints are opinionated windows where layout changes to match space and conventions, and it explicitly names **feed**, **list-detail**, and **supporting pane** as canonical patterns. ⁴⁷

The most web-native way to implement that is:

- **CSS Grid** for macro layout,
- **Flexbox** for one-dimensional sub-layouts,
- **container queries** where the component or pane matters more than viewport size,
- and **content-driven breakpoints** aligned to Material's compact/medium/expanded thinking. 48

```
.app-shell {
  display: grid;
  grid-template-columns: 1fr;
  gap: 24px;
}

@media (min-width: 900px) {
  .app-shell--list-detail {
    grid-template-columns: minmax(280px, 360px) 1fr;
  }
}

@media (min-width: 1280px) {
  .app-shell--supporting-pane {
    grid-template-columns: minmax(280px, 360px) 1fr minmax(280px, 360px);
  }
}

.card-grid {
  display: grid;
  gap: 16px;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
}
```

That is framework-agnostic, matches Material's adaptive layout intent, and avoids the mistake of tying your layout to a single library's component API. 49

Interaction patterns and motion

A strong MD3 web experience uses **state layers**, **clear focus**, and **short, meaningful transitions** instead of large animation flourishes. Official MD3 state guidance provides the state-layer opacity values, and official motion guidance describes fast/default/slow tokens for spatial and effects motion. 50

```
.interactive {
  position: relative;
  border-radius: inherit;
  transition:
    background-color 180ms cubic-bezier(0.2, 0, 0, 1),
    box-shadow 180ms cubic-bezier(0.2, 0, 0, 1),
```

```

    transform 180ms cubic-bezier(0.2, 0, 0, 1);
}

.interactive::after {
  content: "";
  position: absolute;
  inset: 0;
  border-radius: inherit;
  background: currentColor;
  opacity: 0;
  pointer-events: none;
}

.interactive:hover::after { opacity: 0.08; } /* hover */
.interactive:focus-visible::after { opacity: 0.10; } /* focus */
.interactive:active::after { opacity: 0.10; } /* press */

```

For web apps, keep motion tethered to intent:

- use transitions to express **cause and effect**,
- use surfaces/elevation changes to express **layering**,
- never rely on motion alone to convey state,
- and if your component library does not implement MD3 motion tokens directly, define a small internal motion scale in CSS variables and apply it consistently. That last point is especially important because Material Web does not currently expose system motion tokens. ⁵¹

Accessibility and WCAG mapping

Material explicitly treats accessible design as foundational, with guidance on assistive technology, labels, touch targets, and navigation by keyboard or screen reader. The W3C WCAG 2.2 standard remains the conformance anchor for the web. ⁵²

MD3 concern	Practical web requirement	WCAG / WAI anchor
Color roles and contrast	Use official role pairings such as <code>primary/on-primary</code> , <code>surface/on-surface</code> , etc.; do not invent random text/background pairings. ⁵³	WCAG 2.2 overall; Non-text contrast is especially important for controls and focus styles. ⁵⁴
Keyboard interaction	All functionality must be operable by keyboard; map widgets to APG patterns for dialog, menu, tabs, listbox, combobox, and buttons. ⁵⁵	WCAG 2.1.1 Keyboard. ⁵⁶
Visible focus	Preserve a strong <code>:focus-visible</code> indicator; do not remove outlines without a better replacement. ⁵⁷	WCAG 2.4.7 Focus Visible; WCAG 2.2 adds focus-obscured and focus-appearance guidance. ⁵⁸

MD3 concern	Practical web requirement	WCAG / WAI anchor
Name, role, value	Icon-only controls need accessible names; custom widgets must expose state and semantics. Material explicitly says not to include the element role word in the label. ⁵⁹	WCAG 4.1.2 Name, Role, Value. ⁶⁰
Touch targets	Material recommends about 48×48dp targets, which is comfortably larger than WCAG 2.2's 24×24 CSS px minimum for many web cases. ⁶¹	WCAG 2.5.8 Target Size Minimum. ⁶²

A few concrete library notes matter in practice:

- **Material Web dialogs** support focus trapping and recommend labeling dialogs with a headline or `aria-label`. ⁶³
- **Angular Material** says its components are internationalized, accessible, well-tested, and optimized for performance; it also publishes target screen-reader combinations. ²⁸
- **Vuetify** states that components provide keyboard interactions for mouse-based actions and use semantic HTML where applicable. ⁶⁴
- **MUI** positions accessibility as a core value, and recent v9 updates highlight better accessibility defaults and more reliable keyboard navigation. ⁶⁵

Performance, compatibility, testing, and migration

Performance and bundle-size trade-offs

The performance story differs sharply by implementation style.

Material Web publishes explicit bundle-size data. Importing **all** components is listed at **70.9 KB gzip**, while individual button variants are roughly **6–8 KB gzip** and many other components are similarly modular. This strongly rewards **subpath imports** instead of pulling in the whole library. ⁶⁶

MUI emphasizes that bundle size matters, takes snapshots on every commit, and warns that barrel imports like `@mui/material` can significantly slow development startup and rebuilds. Its path-import guidance is not just a micro-optimization; it materially improves DX at scale. ⁶⁷

Angular Material benefits from Angular's package format and secondary entry points, which Angular documents as the unit of code splitting. Angular specifically notes that Angular Material publishes separate logical entry points by component, enabling separate lazy loading where desired. Angular Material's older theming docs also warn that `all-component-themes` emits styles for all components and usually produces unnecessary CSS unless you use the whole library. ⁶⁸

Vuetify documents treeshaking as a major feature and describes it as a way to drastically lower build size by including only the components you use. Its repository README also calls out Vite support and automatic tree-shaking as a bundle-size advantage. ⁶⁹

For React specifically, build-time styling is still a moving target. MUI introduced and documents **Pigment CSS** as a zero-runtime CSS-in-JS path, but the migration guide still calls it early alpha, and MUI's January 2026 update says it is **currently on hold**. Today, the stable expert path in MUI is still **CSS variables + controlled runtime styling**, not betting everything on Pigment CSS yet. ⁷⁰

Cross-browser compatibility

Topic	High-confidence guidance
Material Web browser support	Material Web aims to support the latest two major browser versions and currently lists Chrome 120+, Edge 120+, Firefox 119+, Safari 16.4+, with older Safari potentially needing an ElementInternals polyfill . ⁷¹
MUI browser support	MUI supports the latest stable major browsers and lists Edge 121+, Firefox 121+, Chrome 117+, Safari 17+. It also says no JS polyfill is generally required because unsupported browser features are managed internally. ⁷²
Angular Material browser support	Angular Material supports the most recent two versions of major browsers and publishes screen-reader targets. Angular CLI also uses Browserslist to transform builds appropriately. ⁷³
Vuetify browser support	Vuetify's browser-support docs say Chrome, Firefox, and Safari are supported at least two years back from the major release date; its repo README also notes Safari 13+ with polyfills. ⁷⁴
Popover API	Material Web menus can use native Popover positioning, but the docs explicitly note that Firefox lacked Popover API support at time of writing and the component falls back to fixed positioning. The Popover API itself is now well documented on MDN. ⁷⁵
CSS variables and color-scheme	CSS custom properties are broadly available; <code>color-scheme</code> , <code>prefers-color-scheme</code> , and <code>light-dark()</code> are practical modern tools for MD3 theming. ⁷⁶
Container queries	Container queries are now a pragmatic option for component-level adaptivity and pair well with Material's pane-based layout thinking. ⁷⁷

The expert compatibility rule is simple: **progressively enhance**. Use `@supports` for newer platform features, design reasonable fallbacks, and do not assume that every MD3-inspired interaction can rely on the newest browser primitive. ⁷⁸

Testing and QA

A serious MD3 web implementation needs at least four test layers:

Layer	What to test	Strong tools / official guidance
Unit / component logic	State changes, keyboard handlers, semantic props, focus movement, token mapping.	Angular Material harnesses; MUI testing guidance; Vue Test Utils with Vuetify. ⁷⁹
Visual regression	Light/dark schemes, breakpoints, error states, hover/focus/pressed, density variants.	Playwright screenshots; Storybook visual tests; MUI internally uses Argos. ⁸⁰
Accessibility testing	Labels, roles, focus order, contrast, keyboard reachability, duplicate IDs.	Playwright accessibility testing; Storybook accessibility tests; APG pattern conformance. ⁸¹
Cross-browser E2E	Chromium, Firefox, WebKit with both light and dark modes.	Playwright's cross-browser API. ⁸²

MUI's own testing guide recommends **not** relying heavily on snapshot testing for React component tests, and instead using more user-facing, robust assertions. That advice generalizes well to MD3 work: test **behavior, semantics, and rendered outcomes**, not serialized trees. ⁸³

```
import { test, expect } from '@playwright/test';

test('settings button remains visually stable and accessible', async ({ page }) => {
  await page.goto('/settings');
  await expect(page.getByRole('button', { name: 'Save changes' })).toBeVisible();
  await expect(page).toHaveScreenshot();
});
```

Playwright also recommends prioritizing **role-based locators**, which aligns well with accessibility-first MD3 QA. ⁸⁴

Migration strategies

From legacy MDC-Web

If you are on **MDC-Web**, the strategic answer is to migrate out. The repository is archived, while Material Web is the newer official direction for web components. A pragmatic path is usually:

- freeze visual churn,
- extract design tokens first,
- replace components incrementally by vertical slice,
- and rebuild overlay-heavy pieces carefully because API and DOM assumptions differ. ⁸⁵

From MUI v4 or older MUI styling

MUI's official migration guidance for v4 → v5 emphasizes **codemods** and highlights the major shift from **JSS to Emotion** as one of the biggest changes. For current projects, also account for the v6/v7/v9 shifts around ESM support, browser support, accessibility, and styling direction. ⁸⁶

From Angular Material older themes

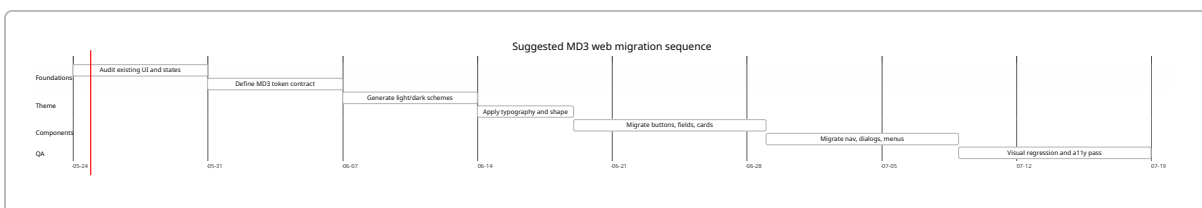
For Angular Material, the important migration mindset is to move toward **M3 palette generation and system variables** rather than carrying forward ad hoc palette overrides. Where possible, standardize on the modern `mat.theme(...)` path and generated theme-color files. ²⁹

From utility-first or non-Material systems

If you are coming from Tailwind, Bootstrap, Chakra, Ant, or a custom in-house system, the safest approach is **not** to swap components first. First define an **MD3 token contract**:

- color roles,
- type roles,
- corner scale,
- elevation levels,
- motion scale,
- state-layer behavior,
- adaptive layout patterns. ⁸⁷

Then migrate components in this sequence:



This sequence is an expert recommendation informed by the official token hierarchy, library migration docs, and current ecosystem status. ⁸⁸

Case studies, pitfalls, reusable patterns, and a mastery roadmap

Real-world examples and case-study references

For web-focused MD3 inspiration, the best official example is the **Material.io redesign** write-up, which explains how dynamic color was used so the site visually reflects the content being consumed. That is the clearest published example of MD3 concepts applied to a real web property. ⁸⁹

For React production inspiration, MUI maintains both a **showcase** of public apps and **example projects** integrated with popular frameworks. Those are especially useful when translating MD principles into production dashboard, landing-page, and e-commerce structures. ⁹⁰

For Vue, Vuetify's **Made with Vuetify** gallery is the best official source of production examples. For broader Material inspiration, Google's **Material Partner Studies** page points to apps such as Lyft, Genius, NPR, Pocket Casts, and Zappos using Material's expressive capabilities. ⁹¹

Common pitfalls and anti-patterns

Pitfall	Why it fails	Better pattern
Branding components directly instead of tokenizing	You lose consistency and make migration hard. Material's architecture is token-first, not per-component hex-first. ⁹²	Define MD3 roles first, then let components consume them.
Using arbitrary text/background color pairs	MD3 role pairings are designed for accessible contrast; random pairings often break contrast and hierarchy. ⁹³	Restrict usage to official role pairings and review exceptions.
Treating elevation as only box-shadow	MD3 elevation also uses tone differences and scrim. ⁹⁴	Combine surface tone, borders, and shadow intentionally.
Removing focus outlines without replacement	Violates Material interaction guidance and WCAG focus requirements. ⁹⁵	Use strong <code>:focus-visible</code> styles with contrast.
Overusing global theme overrides in MUI	MUI documents that the <code>components</code> theme key is not tree-shakable. ⁹⁶	Prefer reusable wrapper components for heavy customization.
Importing MUI barrels everywhere	MUI says this slows startup and rebuilds in development. ⁶⁷	Use path imports.
Using Angular all-component themes by default	Angular Material warns this often emits unnecessary CSS. ⁹⁷	Include only component themes you use when possible.
Starting a new project on MDC-Web	The repo is archived. ²³	Start from Material Web or another actively maintained stack.
Assuming Material Web provides the whole app shell	Material Web's roadmap discussed exploring layouts, but the library remains primarily component-and-token focused. ⁹⁸	Build layout with CSS Grid/Flexbox and your own shell primitives.

Reusable patterns for designers and developers

Designer checklist

- Define a **source color** and approve generated **light and dark schemes** before component design starts. ⁹⁹
- Sign off on **role usage**, not just color swatches: primary, on-primary, surface, on-surface, outline, etc. ¹⁰⁰
- Choose typefaces by mapping **brand** and **plain** fonts to the MD3 type system. ¹⁰¹
- Define a **shape scale** with small, medium, large, full corners. ¹⁰²
- Review layouts in **compact, medium, and expanded** states, not just one desktop artboard. ¹⁰³
- Document hover, focus, pressed, disabled, and drag states for all interactive patterns. ¹⁰⁴
- Confirm labels, target sizes, and semantic intent early. Material explicitly warns against role words in labels and recommends generous touch targets. ¹⁰⁵

Developer checklist

- Publish tokens as **CSS variables** first. ¹⁰⁶
- Keep **component overrides thin**; prefer feeding better tokens into components. ¹⁰
- Support **light/dark** via `color-scheme`, `prefers-color-scheme`, or your framework's color-scheme API. ¹⁰⁷
- Add **focus-visible** styles and test keyboard navigation against APG patterns. ¹⁰⁸
- Test all high-value components in **Chromium, Firefox, and WebKit**. ⁸²
- Run visual regression on **light, dark, high-contrast if supported, error states, and responsive states**. ¹⁰⁹
- For React, avoid barrel imports; for Angular, avoid all-component CSS; for web components, import only what you use. ¹¹⁰

A ten-week learning and practice roadmap

Timeframe	Focus	Milestones	Deliverables
Weeks 1–2	Learn the MD3 spec deeply	Understand token hierarchy, color roles, typography roles, shape, elevation, states, motion, adaptive layout. ¹¹¹	A written MD3 glossary, one token map, one layout audit of an existing site.
Weeks 3–4	Build a framework-agnostic token system	Publish light/dark roles as CSS variables and implement 6–8 primitives. ²²	Token package, demo page, button/card/field/list/dialog primitives.
Weeks 5–6	Implement one framework adapter	Choose one: Material Web, MUI, Angular Material, or Vuetify. Use real theming APIs. ¹¹²	A themed starter app in your chosen stack.
Weeks 7–8	Layout, accessibility, and motion	Add canonical layouts, keyboard support, visible focus, state layers, motion policy. ¹¹³	Responsive list-detail app shell plus accessibility review checklist.

Timeframe	Focus	Milestones	Deliverables
Weeks 9–10	QA and migration drills	Add Playwright, visual regression, a11y checks, and migrate one legacy screen. ¹¹⁴	CI test suite, screenshot baselines, migration cookbook, before/after diff deck.

By the end of this roadmap, the skill target is not “I know Material components.” It is “I can design and implement a tokenized MD3 system, adapt it to different frameworks, and verify it under accessibility, performance, and compatibility constraints.” That is the real expert threshold. This is a synthesis of the official MD3 model and the implementation ecosystems documented above. ¹¹⁵

Open questions and limitations

A few official MD3 pages on **m3.material.io** are heavily JavaScript-driven, so some fine-grained spec details in this report are based on the official search snippets and linked official resources rather than fully parsed page text. The core findings are still high confidence, but if you need exhaustive token-by-token values for every component, you should cross-check the live component pages in the official docs. ¹¹⁶

The ecosystem is also still moving. In particular, **Material Web’s maintenance-mode notice**, **MUI’s lack of native MD3 defaults**, and **MUI Pigment CSS being on hold** are current-status items that can change with future releases. ¹¹⁷

¹ <https://m3.material.io/>

<https://m3.material.io/>

² ²⁴ ²⁵ ²⁷ ⁴³ <https://mui.com/material-ui/getting-started/>

<https://mui.com/material-ui/getting-started/>

³ ¹¹⁶ <https://m3.material.io/foundations/design-tokens>

<https://m3.material.io/foundations/design-tokens>

⁴ ⁵ ⁸ ⁹ ¹⁰ ²² ³⁴ ⁴² ⁸⁷ ⁸⁸ ⁹² ¹⁰⁶ ¹¹¹ ¹¹⁵ <https://material-web.dev/theming/material-theming/>

<https://material-web.dev/theming/material-theming/>

⁶ ²¹ ³⁵ ¹¹² ¹¹⁷ <https://github.com/material-components/material-web>

<https://github.com/material-components/material-web>

⁷ <https://m3.material.io/components>

<https://m3.material.io/components>

¹¹ <https://m3.material.io/styles/color/system/how-the-system-works>

<https://m3.material.io/styles/color/system/how-the-system-works>

¹² ⁴¹ ⁹⁹ <https://m3.material.io/blog/science-of-color-design>

<https://m3.material.io/blog/science-of-color-design>

¹³ ⁴⁰ ⁵³ <https://material-web.dev/theming/color/>

<https://material-web.dev/theming/color/>

- 14 <https://m3.material.io/styles/typography/applying-type>
<https://m3.material.io/styles/typography/applying-type>
- 15 <https://m3.material.io/styles/shape>
<https://m3.material.io/styles/shape>
- 16 <https://m3.material.io/styles/elevation>
<https://m3.material.io/styles/elevation>
- 17 50 104 <https://m3.material.io/foundations/interaction/states/state-layers>
<https://m3.material.io/foundations/interaction/states/state-layers>
- 18 <https://m3.material.io/styles/motion/overview/how-it-works>
<https://m3.material.io/styles/motion/overview/how-it-works>
- 19 49 <https://m3.material.io/foundations/adaptive-design/overview>
<https://m3.material.io/foundations/adaptive-design/overview>
- 20 <https://material-web.dev/>
<https://material-web.dev/>
- 23 85 <https://github.com/material-components/material-components-web>
<https://github.com/material-components/material-components-web>
- 26 <https://mui.com/material-ui/customization/css-theme-variables/usage/>
<https://mui.com/material-ui/customization/css-theme-variables/usage/>
- 28 73 <https://github.com/angular/components>
<https://github.com/angular/components>
- 29 30 36 45 <https://github.com/angular/components/blob/main/src/material/schematics/ng-generate/theme-color/README.md>
<https://github.com/angular/components/blob/main/src/material/schematics/ng-generate/theme-color/README.md>
- 31 33 <https://github.com/vuetifyjs/vuetify>
<https://github.com/vuetifyjs/vuetify>
- 32 46 <https://vuetifyjs.com/en/features/theme/>
<https://vuetifyjs.com/en/features/theme/>
- 37 <https://github.com/material-foundation/material-color-utils>
<https://github.com/material-foundation/material-color-utils>
- 38 39 <https://mui.com/material-ui/customization/css-theme-variables/overview/>
<https://mui.com/material-ui/customization/css-theme-variables/overview/>
- 44 96 <https://mui.com/material-ui/customization/theme-components/>
<https://mui.com/material-ui/customization/theme-components/>
- 47 103 <https://m3.material.io/foundations/layout/applying-layout>
<https://m3.material.io/foundations/layout/applying-layout>
- 48 77 https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Containment/Container_queries
https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Containment/Container_queries
- 51 <https://m3.material.io/styles/motion/transitions>
<https://m3.material.io/styles/motion/transitions>

- 52 <https://m3.material.io/foundations/designing/overview>
<https://m3.material.io/foundations/designing/overview>
- 54 <https://www.w3.org/TR/WCAG22/>
<https://www.w3.org/TR/WCAG22/>
- 55 <https://m3.material.io/foundations/designing/flow>
<https://m3.material.io/foundations/designing/flow>
- 56 <https://www.w3.org/WAI/WCAG21/Understanding/keyboard.html>
<https://www.w3.org/WAI/WCAG21/Understanding/keyboard.html>
- 57 95 <https://m3.material.io/foundations/interaction/states>
<https://m3.material.io/foundations/interaction/states>
- 58 108 <https://www.w3.org/WAI/WCAG22/Understanding/focus-visible.html>
<https://www.w3.org/WAI/WCAG22/Understanding/focus-visible.html>
- 59 105 <https://m3.material.io/foundations/designing/elements>
<https://m3.material.io/foundations/designing/elements>
- 60 <https://www.w3.org/WAI/WCAG22/Understanding/name-role-value>
<https://www.w3.org/WAI/WCAG22/Understanding/name-role-value>
- 61 <https://m3.material.io/foundations/designing/structure>
<https://m3.material.io/foundations/designing/structure>
- 62 <https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html>
<https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html>
- 63 <https://material-web.dev/components/dialog/>
<https://material-web.dev/components/dialog/>
- 64 <https://vuetifyjs.com/en/features/accessibility/>
<https://vuetifyjs.com/en/features/accessibility/>
- 65 <https://mui.com/>
<https://mui.com/>
- 66 <https://material-web.dev/about/size/>
<https://material-web.dev/about/size/>
- 67 110 <https://mui.com/material-ui/guides/minimizing-bundle-size/>
<https://mui.com/material-ui/guides/minimizing-bundle-size/>
- 68 <https://angular.dev/tools/libraries/angular-package-format>
<https://angular.dev/tools/libraries/angular-package-format>
- 69 <https://vuetifyjs.com/en/features/treeshaking/>
<https://vuetifyjs.com/en/features/treeshaking/>
- 70 <https://mui.com/material-ui/experimental-api/pigment-css/>
<https://mui.com/material-ui/experimental-api/pigment-css/>
- 71 <https://material-web.dev/about/support/>
<https://material-web.dev/about/support/>

- 72 <https://mui.com/material-ui/getting-started/supported-platforms/>
<https://mui.com/material-ui/getting-started/supported-platforms/>
- 74 <https://vuetifyjs.com/en/getting-started/browser-support/>
<https://vuetifyjs.com/en/getting-started/browser-support/>
- 75 <https://material-web.dev/components/menu/>
<https://material-web.dev/components/menu/>
- 76 <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/--%2A>
<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/--%2A>
- 78 <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/%40supports>
<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/%40supports>
- 79 <https://v9.material.angular.dev/cdk/test-harnesses>
<https://v9.material.angular.dev/cdk/test-harnesses>
- 80 109 114 <https://playwright.dev/docs/test-snapshots>
<https://playwright.dev/docs/test-snapshots>
- 81 <https://playwright.dev/docs/accessibility-testing>
<https://playwright.dev/docs/accessibility-testing>
- 82 <https://playwright.dev/>
<https://playwright.dev/>
- 83 <https://mui.com/material-ui/guides/testing/>
<https://mui.com/material-ui/guides/testing/>
- 84 <https://playwright.dev/docs/locators>
<https://playwright.dev/docs/locators>
- 86 <https://mui.com/material-ui/migration/migration-v4/>
<https://mui.com/material-ui/migration/migration-v4/>
- 89 <https://m3.material.io/blog/material-io-redesign>
<https://m3.material.io/blog/material-io-redesign>
- 90 <https://mui.com/material-ui/discover-more/showcase/>
<https://mui.com/material-ui/discover-more/showcase/>
- 91 <https://vuetifyjs.com/en/resources/made-with-vuetify/>
<https://vuetifyjs.com/en/resources/made-with-vuetify/>
- 93 100 <https://m3.material.io/styles/color/roles>
<https://m3.material.io/styles/color/roles>
- 94 <https://m3.material.io/styles/elevation/applying-elevation>
<https://m3.material.io/styles/elevation/applying-elevation>
- 97 <https://v14.material.angular.dev/docs-content/guides/theming>
<https://v14.material.angular.dev/docs-content/guides/theming>
- 98 <https://material-web.dev/about/roadmap/>
<https://material-web.dev/about/roadmap/>

101 <https://material-web.dev/theming/typography/>

<https://material-web.dev/theming/typography/>

102 <https://material-web.dev/theming/shape/>

<https://material-web.dev/theming/shape/>

107 <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/color-scheme>

<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/color-scheme>

113 <https://m3.material.io/foundations/adaptive-design/canonical-layouts>

<https://m3.material.io/foundations/adaptive-design/canonical-layouts>